INSTITUTO POLITÉCNICO DO CÁVADO E DO AVE

ESCOLA SUPERIOR DE TECNOLOGIA

Licenciatura em Engenharia de Sistemas Informáticos
Unidade Curricular de Integração de Sistemas de Informação
2026/27

Trabalho Prático I

ISI Store — Loja E-commerce

Processo *Extract, Transform and Load* (ETL)

Autor: A PREENCHER
Número: A PREENCHER
Email: A PREENCHER
Repositório Git: `A-PREENCHER-LINK-GIT`

17 de outubro de 2026

Resumo

O presente relatório documenta o processo de *Extract, Transformation and Load* (ETL) desenvolvido para a loja fictícia ISI Store. O problema consistiu na dispersão de dados por sistemas isolados: vendas em CSV, clientes em JSON, catálogo em XML, *stock* em base de dados relacional PostgreSQL e catálogo externo via API REST. No estado da arte compararam-se as plataformas Pentaho Data Integration, Talend Open Studio e KNIME, tendo-se selecionado o Pentaho pela orquestração visual de transformações e *jobs* (Hitachi Vantara, 2026). A abordagem implementou extração multiformato, limpeza com expressões regulares, anonimização, junções e agregações, carga para *Data Mart*, exportações CSV/JSON, notificações por correio eletrónico, transferência FTP e painel complementar em Node-RED com MQTT (Node-RED, 2026). Obteve-se execução integral válida: 13 vendas reduziram-se a 10 limpas, 6 clientes a 2 válidos, 158 produtos integrados, 10 linhas carregadas em `dw_vendas` e agregação por cidade (Braga 2072/3, Porto 948/2, nulo 429/5). Concluiu-se que o processo cumpre as três fases obrigatórias do enunciado (Ferreira e Ribeiro, 2026).

Palavras-chave: ETL; Pentaho; Node-RED; MQTT; PostgreSQL; Expressões Regulares.

Siglas

No presente documento utilizam-se as siglas abaixo, ordenadas alfabeticamente.

- BD – Base de Dados
- CSV – *Comma-Separated Values*
- ETL – *Extract, Transformation and Load*
- FTP – *File Transfer Protocol*
- JSON – *JavaScript Object Notation*
- MQTT – *Message Queuing Telemetry Transport*
- REST – *Representational State Transfer*
- RGPD – Regulamento Geral sobre a Proteção de Dados
- XML – *eXtensible Markup Language*

Índice

1 Enquadramento **6**
1.1 Equipa e contexto 6
1.2 Objetivos 6
1.3 Metodologia de trabalho 6
1.4 Estrutura do documento 6

2 Problema **7**

3 Estratégia utilizada **8**
3.1 Soluções existentes 8
3.2 Arquitetura adotada 8

4 Transformações **10**
4.1 E_extracao.ktr 10
4.2 T_limpeza.ktr 16
4.3 T_clientes.ktr 17
4.4 T_joins.ktr 18

5 Jobs **20**
5.1 J_principal.kjb 20
5.2 Persistência e exportação 20
5.3 Node-RED e painel 21

6 Vídeo com demonstração (QR Code) **23**

7 Conclusão e Trabalhos futuros **24**

8 Referências **25**

A Contagens de validação **26**

Lista de Figuras

1 Fase E no Spoon – extrações paralelas: CSV, JSON, XML, Generate+REST e Table Input. Fonte: autor (2026). . . . 10
2 Log de execução E_extracao – 13 vendas, 6 clientes, 158 produtos, 1 REST e 158 stock. Fonte: autor (2026). . . . 10
3 Preview vendas.csv – 13 linhas com quantidade 0, duplicado V002, id_cliente null e data errada. Fonte: autor (2026). . . . 11
4 Preview clientes.json – 6 linhas com email inválido, email null, NIF ABC123 e duplicado. Fonte: autor (2026). . . . 11
5 Preview produtos.xml – 158 produtos (P001-P158) com Loop XPath /produtos/produto. Fonte: autor (2026). . . . 12
6 Preview REST client – 1 linha status 200 com JSON FakeStore (trigger Generate Rows dummy=1). Fonte: autor (2026). . . . 12
7 Preview Table Input – 158 linhas da tabela stock. Fonte: autor (2026). . . . 13
8 Tabelas dw_vendas e stock na BD isi_store. Fonte: autor (2026). . . . 14
9 Conteúdo da tabela stock. Fonte: autor (2026). . . . 15
10 Tabela dw_vendas vazia antes da carga. Fonte: autor (2026). . . . 15
11 Conexão PostgreSQL isi_store testada com sucesso. Fonte: autor (2026). . . . 16
12 Filter Rows vendas – 10 linhas após remover quantidade 0 e id_cliente null. Fonte: autor (2026). . . . 16
13 Calculator final vendas – 10 linhas com total_linha correto (V001 1799.98). Fonte: autor (2026). . . . 17
14 Replace telemóvel – normalização com Regex [\s-] (912 345 678 para 912345678). Fonte: autor (2026). . . . 17
15 Validação Regex – 6 clientes com email_ok, telemovel_ok e nif_ok (2 limpos C001/C003). Fonte: autor (2026). . . . 17
16 Clientes limpos – 2 válidos após filtro Y/Y/Y com schema remapeado. Fonte: autor (2026). 18
17 Anonimização RGPD – nif_anon ******789 e email_anon a***@gmail. Fonte: autor (2026). . . . 18
18 Join 1 Stream Lookup vendas+clientes – 10 linhas, nulls por integridade referencial. Fonte: autor (2026). . . . 18
19 Join final Stream Lookup 2 +produtos – 10 linhas com nome_prod e categoria_prod. Fonte: autor (2026). . . . 18
20 Canvas T_join completo – vendas+clientes+produtos, 2 lookups, Sort+Group. Fonte: autor (2026). . . . 19
21 Group By cidade_cliente – 3 linhas: null 429/5, Braga 2072/3, Porto 948/2. Fonte: autor (2026). . . . 19
22 Job principal – E, T, L, FTP csv/json, Mail e Write to Log com sucesso. Fonte: autor (2026). 20
23 Correio eletrónico de alerta com totais por cidade. Fonte: autor (2026). . . . 20
24 Tabela dw_vendas com 10 linhas e totais corretos (V001 1799.98). Fonte: autor (2026). . 21
25 Ficheiros exportados em Dados_Saida. Fonte: autor (2026). . . . 21
26 Fluxos Node-RED – streaming isi/vendas e painel. Fonte: autor (2026). . . . 22
27 Painel com totais Braga 2072, Porto 948 e nulo 429. Fonte: autor (2026). . . . 22

Índice de Códigos

1 Criação das tabelas dw_vendas e stock na BD isi_store. Fonte: autor (2026). 9

1 Enquadramento

O presente capítulo apresenta o contexto do trabalho. A Secção 1.1 identifica a equipa e o contexto curricular, a Secção 1.2 define os objetivos, a Secção 1.3 descreve a metodologia de trabalho e a Secção 1.4 apresenta a estrutura do documento.

1.1 Equipa e contexto

O trabalho foi desenvolvido individualmente no âmbito da Unidade Curricular de Integração de Sistemas de Informação (ISI), da Licenciatura em Engenharia de Sistemas Informáticos, no ano letivo 2026/27. Decidiu-se explorar o tema ISI Store, uma loja fictícia de informática e mobiliário de escritório.

1.2 Objetivos

Definiram-se objetivos poucos e claros: (i) consolidar conceitos de integração de sistemas de informação com dados; (ii) especificar um cenário de aplicação de processos ETL; (iii) explorar o Pentaho Data Integration com ferramenta complementar Node-RED; (iv) documentar o processo com qualidade (Ferreira e Ribeiro, 2026).

1.3 Metodologia de trabalho

Seguiu-se uma metodologia por fases: especificação das fontes heterogéneas, implementação incremental E–T–L no Pentaho, validação por contagens e *previews*, orquestração em *Job* com correio eletrónico e FTP, integração complementar via MQTT e redação progressiva do relatório.

1.4 Estrutura do documento

A Secção 2 descreve o problema, a Secção 3 a estratégia e o enquadramento teórico e prático, a Secção 4 as transformações, a Secção 5 os *jobs* e destinos, a Secção 6 o vídeo de demonstração, a Secção 7 a conclusão e a Secção 8 as referências.

Plataforma principal: Pentaho Data Integration (Kettle / Spoon) Community Edition 10.2. Ferramentas complementares: PostgreSQL com DBeaver, Node-RED com Mosquitto e painel Node-RED Dashboard. Repositório Git: `A-PREENCHER-LINK-GIT`.

2 Problema

O presente capítulo explora o problema, identifica a ausência de soluções e releva a pertinência do contexto atual. A loja ISI Store mantinha sistemas isolados: vendas da loja física em CSV, registo de clientes em JSON, catálogo em XML e *stock* em PostgreSQL, sem visão consolidada. Verificou-se ausência de solução de integração, com dados sujos: duplicado V002, quantidade zero (V005), cliente nulo (V007) e data com formato errado (V009 com `/`), correio eletrónico inválido e NIF não numérico.

Objetivo: construir um processo ETL completo que integre estas fontes, limpe e valide com expressões regulares, efetue junções e agregações, carregue um *Data Mart* e gere alertas e painel de suporte à decisão (Ferreira e Ribeiro, 2026).

3 Estratégia utilizada

O presente capítulo identifica os operadores e processos envolvidos. A Secção 3.1 compara as soluções existentes e a Secção 3.2 apresenta a arquitetura adotada.

3.1 Soluções existentes

Identificaram-se soluções no domínio da integração de dados: Pentaho Data Integration, Talend Open Studio e KNIME. A Tabela 1 compara-as segundo critérios alinhados com o trabalho: orquestração visual de transformações e *jobs*, diversidade de conectores e integração complementar. Concluiu-se que o Pentaho se adequava melhor por combinar transformações e *jobs* com correio eletrónico e FTP na mesma ferramenta visual, enquanto o Talend exige maior configuração e o KNIME se orienta mais para *analytics* (Hitachi Vantara, 2026; Talend, 2026; KNIME, 2026).

Critério	Pentaho 10.2	Talend	KNIME
Orquestração ETL + Job visual	Sim	Parcial	Parcial
Conectores CSV/JSON/XML/REST/BD	Sim	Sim	Sim
Email + FTP nativos no Job	Sim	Parcial	Não

Tabela 1: Quadro comparativo de plataformas. Fonte: autor (2026).

3.2 Arquitetura adotada

A arquitetura seguiu as três fases obrigatórias, sintetizadas na Tabela 2: fase E multiformato, fase T com limpeza, expressões regulares, junções e agregações, e fase L com persistência, exportações, notificações e painel. As expressões regulares utilizadas constam da Tabela 3 e as tabelas de destino no Código 1.

Fonte	Formato	Step Pentaho
vendas.csv (ficheiro local)	CSV	CSV File Input
clientes.json (ficheiro local)	JSON	JSON Input
produtos.xml (ficheiro local)	XML	Get Data from XML
API FakeStore	JSON REST	Generate Rows + REST Client
stock BD PostgreSQL	Tabela SQL	Table Input

Tabela 2: Fase E – diversidade de formatos e fontes. Fonte: autor (2026).

Fase T: Sort Rows, Unique Rows, Filter Rows, Regex Evaluation, Replace in String, Calculator, Stream Lookup, Group By e Write to Log.

Fase L: Table Output (dw_vendas), Text File Output (CSV) e JSON Output, Send Mail, FTP Put e Write to Log. Orquestração via Job.

Regex utilizadas (Tabela 3):

Campo	Expressão regular
Email	`^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$`
Telemóvel PT após limpar `[\s-]`	`^9[1236][0-9]{7}$`
NIF	`^[0-9]{9}$`
Data	`^\d{4}-\d{2}-\d{2}$`

Tabela 3: Expressões regulares aplicadas. Fonte: autor (2026).

Criação das tabelas de destino:

```sql
CREATE TABLE dw_vendas (
  id_venda VARCHAR(10) PRIMARY KEY,
  id_cliente VARCHAR(10),
  id_produto VARCHAR(10),
  data DATE,
  quantidade INT,
  preco_unit NUMERIC(10,2),
  total_linha NUMERIC(10,2)
);
CREATE TABLE stock (
  id_produto VARCHAR(10) PRIMARY KEY,
  stock_atual INT
);
```

Código 1: Criação das tabelas dw_vendas e stock na BD isi_store. Fonte: autor (2026).

4 Transformações

O presente capítulo descreve as transformações implementadas. A Secção 4.1 apresenta a extração, a Secção 4.2 a limpeza de vendas, a Secção 4.3 a validação de clientes e a Secção 4.4 as junções e agregações.

4.1 E_extracao.ktr

Implementou-se leitura paralela antes do processamento principal, com *schemas* heterogéneos preservados (a junção efetuou-se na fase T). O canvas com os 6 *steps* paralelos encontra-se na Figura 1 e o log de execução na Figura 2: vendas W=13, clientes W=6, produtos W=158, REST W=1 e *stock* W=158.

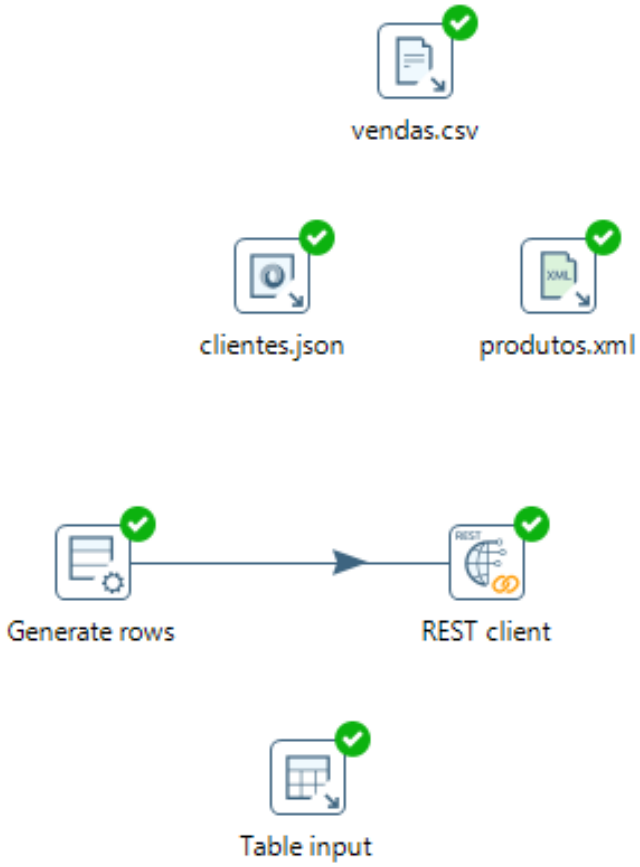


Figura 1: Fase E no Spoon – extrações paralelas: CSV, JSON, XML, Generate+REST e Table Input. Fonte: autor (2026).

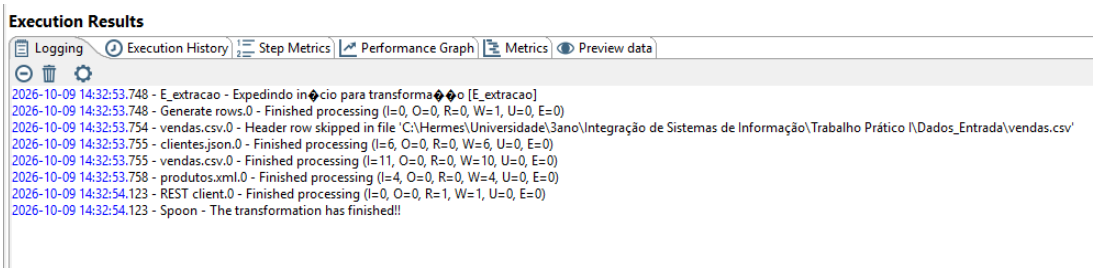
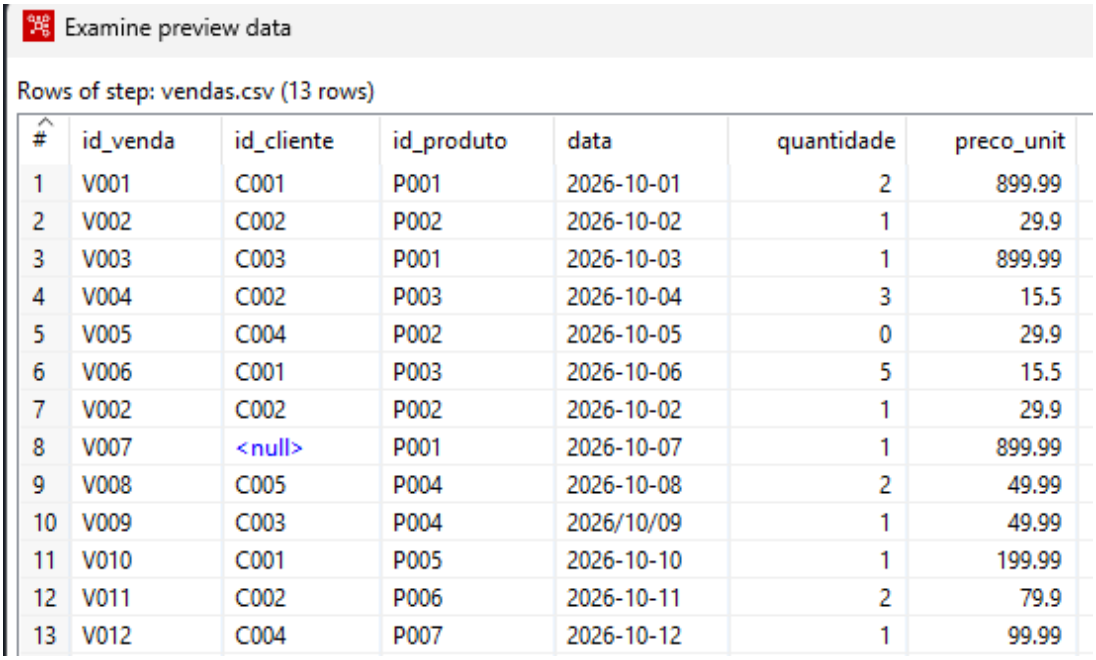


Figura 2: Log de execução E_extracao – 13 vendas, 6 clientes, 158 produtos, 1 REST e 158 stock. Fonte: autor (2026).

Os dados de entrada evidenciam os problemas a tratar: a Figura 3 mostra as 13 vendas com quantidade zero, duplicado V002, cliente nulo e data errada; a Figura 4 mostra os 6 clientes com correio inválido, correio nulo, NIF não numérico e duplicado; a Figura 5 mostra os 158 produtos com *Loop XPath*; a Figura 6 mostra a resposta 200 da API FakeStore obtida com *trigger* Generate Rows; e a Figura 7 mostra as 158 linhas de *stock*.


Examine preview data

Rows of step: vendas.csv (13 rows)

#	id_venda	id_cliente	id_produto	data	quantidade	preco_unit
1	V001	C001	P001	2026-10-01	2	899.99
2	V002	C002	P002	2026-10-02	1	29.9
3	V003	C003	P001	2026-10-03	1	899.99
4	V004	C002	P003	2026-10-04	3	15.5
5	V005	C004	P002	2026-10-05	0	29.9
6	V006	C001	P003	2026-10-06	5	15.5
7	V002	C002	P002	2026-10-02	1	29.9
8	V007	<null>	P001	2026-10-07	1	899.99
9	V008	C005	P004	2026-10-08	2	49.99
10	V009	C003	P004	2026/10/09	1	49.99
11	V010	C001	P005	2026-10-10	1	199.99
12	V011	C002	P006	2026-10-11	2	79.9
13	V012	C004	P007	2026-10-12	1	99.99

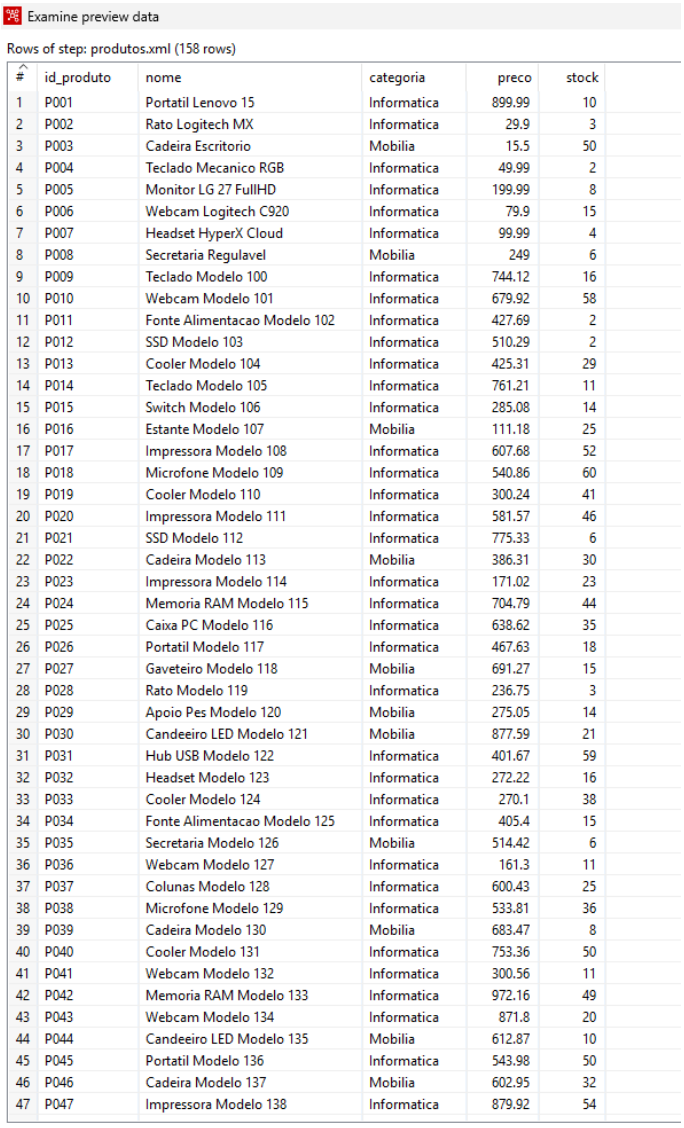
Figura 3: Preview vendas.csv – 13 linhas com quantidade 0, duplicado V002, id_cliente null e data errada. Fonte: autor (2026).


Examine preview data

Rows of step: clientes.json (6 rows)

#	id_cliente	email	nome	telemovel	nif	cidade
1	C001	ana.silva@gmail.com	ana silva	912 345 678	123456789	Braga
2	C002	joao.pereira@mail	JOAO PEREIRA	913-456-789	987654321	Guimaraes
3	C003	maria.costa@hotmail.com	Maria Costa	914456789	456789123	Porto
4	C004	<null>	Pedro Santos	915 456 789	321654987	Braga
5	C005	ines.rocha@gmail.com	Ines Rocha	91645678	ABC123	Lisboa
6	C002	joao.pereira@mail	JOAO PEREIRA	913-456-789	987654321	Guimaraes

Figura 4: Preview clientes.json – 6 linhas com email inválido, email null, NIF ABC123 e duplicado. Fonte: autor (2026).



#	id_produto	nome	categoria	preco	stock
1	P001	Portatil Lenovo 15	Informatica	899.99	10
2	P002	Rato Logitech MX	Informatica	29.9	3
3	P003	Cadeira Escritorio	Mobilia	15.5	50
4	P004	Teclado Mecanico RGB	Informatica	49.99	2
5	P005	Monitor LG 27 FullHD	Informatica	199.99	8
6	P006	Webcam Logitech C920	Informatica	79.9	15
7	P007	Headset HyperX Cloud	Informatica	99.99	4
8	P008	Secretaria Regulavel	Mobilia	249	6
9	P009	Teclado Modelo 100	Informatica	744.12	16
10	P010	Webcam Modelo 101	Informatica	679.92	58
11	P011	Fonte Alimentacao Modelo 102	Informatica	427.69	2
12	P012	SSD Modelo 103	Informatica	510.29	2
13	P013	Cooler Modelo 104	Informatica	425.31	29
14	P014	Teclado Modelo 105	Informatica	761.21	11
15	P015	Switch Modelo 106	Informatica	285.08	14
16	P016	Estante Modelo 107	Mobilia	111.18	25
17	P017	Impressora Modelo 108	Informatica	607.68	52
18	P018	Microfone Modelo 109	Informatica	540.86	60
19	P019	Cooler Modelo 110	Informatica	300.24	41
20	P020	Impressora Modelo 111	Informatica	581.57	46
21	P021	SSD Modelo 112	Informatica	775.33	6
22	P022	Cadeira Modelo 113	Mobilia	386.31	30
23	P023	Impressora Modelo 114	Informatica	171.02	23
24	P024	Memoria RAM Modelo 115	Informatica	704.79	44
25	P025	Caixa PC Modelo 116	Informatica	638.62	35
26	P026	Portatil Modelo 117	Informatica	467.63	18
27	P027	Gaveteiro Modelo 118	Mobilia	691.27	15
28	P028	Rato Modelo 119	Informatica	236.75	3
29	P029	Apoio Pes Modelo 120	Mobilia	275.05	14
30	P030	Candeeiro LED Modelo 121	Mobilia	877.59	21
31	P031	Hub USB Modelo 122	Informatica	401.67	59
32	P032	Headset Modelo 123	Informatica	272.22	16
33	P033	Cooler Modelo 124	Informatica	270.1	38
34	P034	Fonte Alimentacao Modelo 125	Informatica	405.4	15
35	P035	Secretaria Modelo 126	Mobilia	514.42	6
36	P036	Webcam Modelo 127	Informatica	161.3	11
37	P037	Colunas Modelo 128	Informatica	600.43	25
38	P038	Microfone Modelo 129	Informatica	533.81	36
39	P039	Cadeira Modelo 130	Mobilia	683.47	8
40	P040	Cooler Modelo 131	Informatica	753.36	50
41	P041	Webcam Modelo 132	Informatica	300.56	11
42	P042	Memoria RAM Modelo 133	Informatica	972.16	49
43	P043	Webcam Modelo 134	Informatica	871.8	20
44	P044	Candeeiro LED Modelo 135	Mobilia	612.87	10
45	P045	Portatil Modelo 136	Informatica	543.98	50
46	P046	Cadeira Modelo 137	Mobilia	602.95	32
47	P047	Impressora Modelo 138	Informatica	879.92	54

Figura 5: Preview produtos.xml – 158 produtos (P001-P158) com Loop XPath /produtos/produto. Fonte: autor (2026).

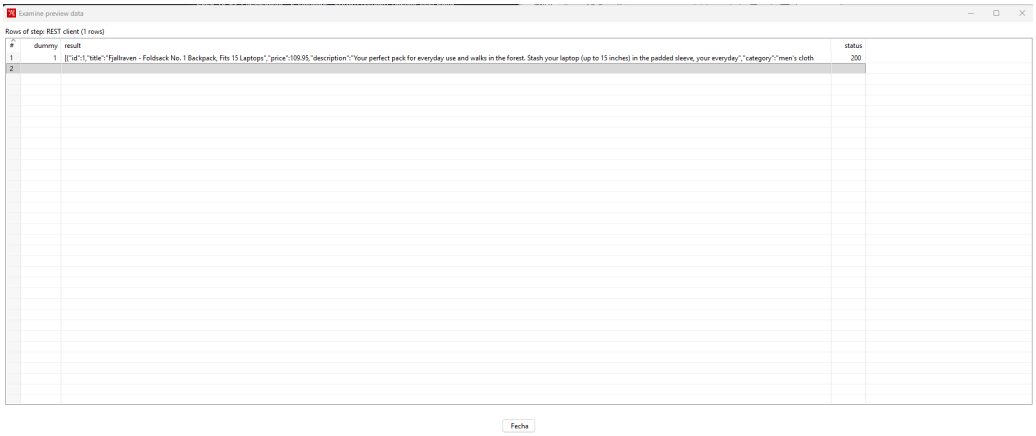
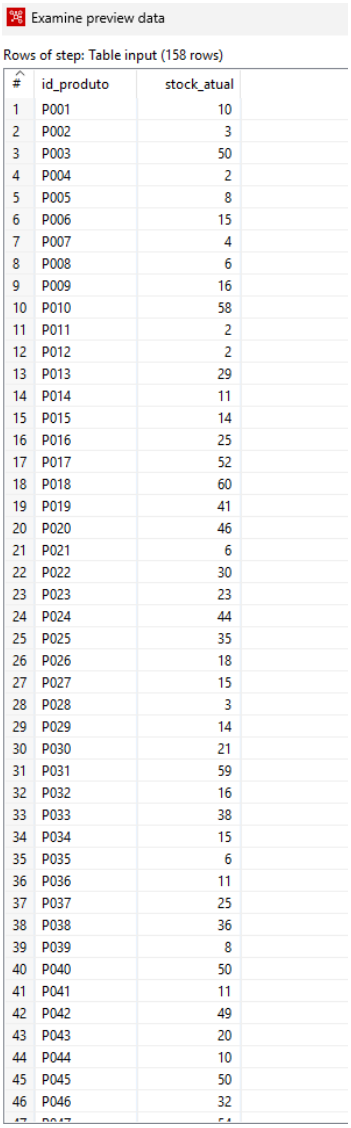


Figura 6: Preview REST client – 1 linha status 200 com JSON FakeStore (trigger Generate Rows dummy=1). Fonte: autor (2026).



#	id_produto	stock_atual
1	P001	10
2	P002	3
3	P003	50
4	P004	2
5	P005	8
6	P006	15
7	P007	4
8	P008	6
9	P009	16
10	P010	58
11	P011	2
12	P012	2
13	P013	29
14	P014	11
15	P015	14
16	P016	25
17	P017	52
18	P018	60
19	P019	41
20	P020	46
21	P021	6
22	P022	30
23	P023	23
24	P024	44
25	P025	35
26	P026	18
27	P027	15
28	P028	3
29	P029	14
30	P030	21
31	P031	59
32	P032	16
33	P033	38
34	P034	15
35	P035	6
36	P036	11
37	P037	25
38	P038	36
39	P039	8
40	P040	50
41	P041	11
42	P042	49
43	P043	20
44	P044	10
45	P045	50
46	P046	32

Figura 7: Preview Table Input – 158 linhas da tabela stock. Fonte: autor (2026).

A base de dados `isi_store` encontra-se documentada nas Figuras 8 a 11: tabelas criadas, conteúdo de *stock*, `dw_vendas` vazia antes da carga e conexão testada com sucesso.

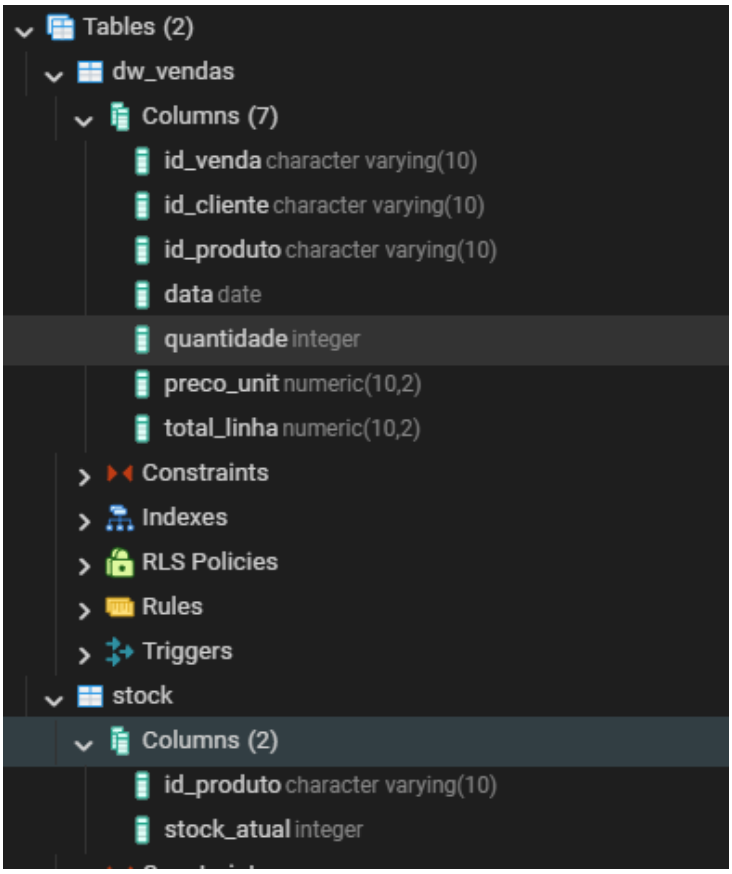


Figura 8: Tabelas dw_vendas e stock na BD isi_store. Fonte: autor (2026).

	id_produto [PK] character varying (10)	stock_atual integer
1	P001	10
2	P002	3
3	P003	50
4	P004	2
5	P005	8
6	P006	15
7	P007	4
8	P008	6
9	P009	16
10	P010	58

Figura 9: Conteúdo da tabela stock. Fonte: autor (2026).

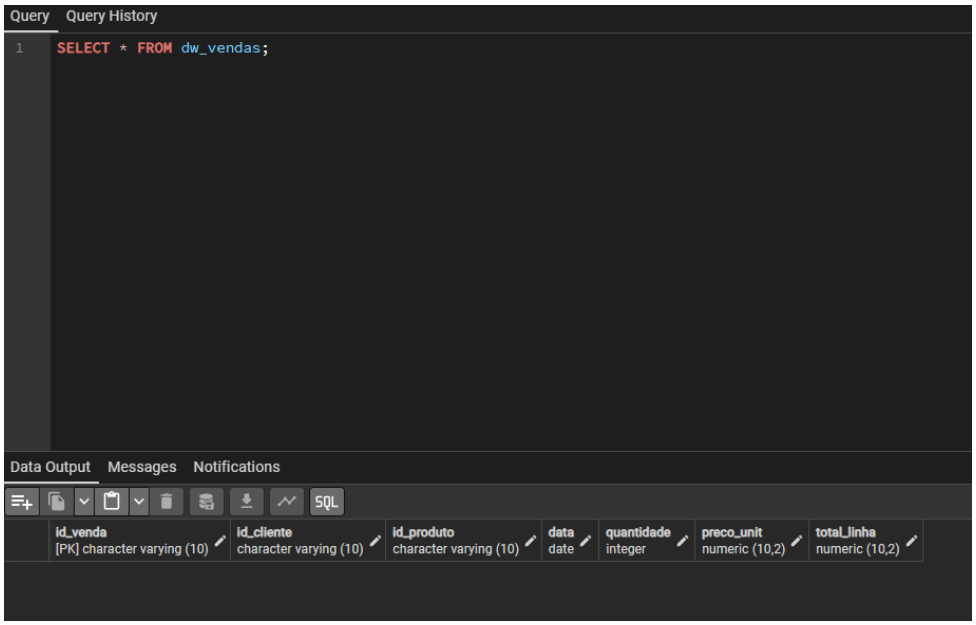


Figura 10: Tabela dw_vendas vazia antes da carga. Fonte: autor (2026).

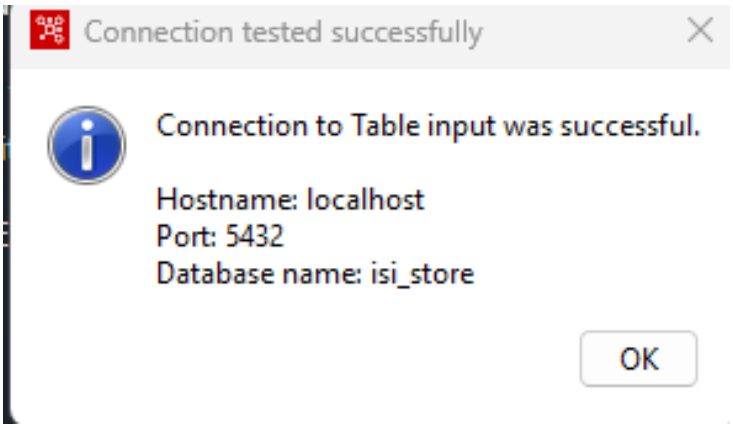


Figura 11: Conexão PostgreSQL isi_store testada com sucesso. Fonte: autor (2026).

4.2 T_limpeza.ktr

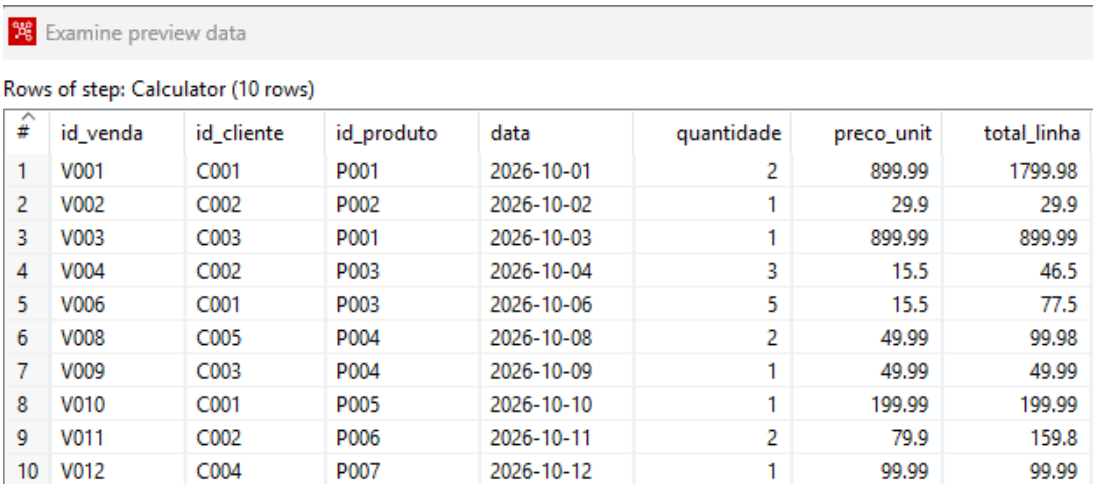
Aplicou-se Sort Rows por `id_venda` seguido de Unique Rows, que removeu o duplicado V002 (13 para 12), e Filter Rows com `quantidade>0 AND id_cliente IS NOT NULL`, que removeu V005 e V007 (12 para 10), conforme a Figura 12. Efetuou-se Replace da data de `/` para `-` (V009) e calculou-se `total_linha=quantidade*preco_unit` em Calculator BigNumber com precisão 2, após correção do *locale* (Decimal `.` sem agrupamento), conforme a Figura 13 com V001 a 1799.98.

Examine preview data

Rows of step: Filter rows (10 rows)

#	id_venda	id_cliente	id_produto	data	quantidade	preco_unit
1	V001	C001	P001	2026-10-01	2	899.99
2	V002	C002	P002	2026-10-02	1	29.9
3	V003	C003	P001	2026-10-03	1	899.99
4	V004	C002	P003	2026-10-04	3	15.5
5	V006	C001	P003	2026-10-06	5	15.5
6	V008	C005	P004	2026-10-08	2	49.99
7	V009	C003	P004	2026/10/09	1	49.99
8	V010	C001	P005	2026-10-10	1	199.99
9	V011	C002	P006	2026-10-11	2	79.9
10	V012	C004	P007	2026-10-12	1	99.99

Figura 12: Filter Rows vendas – 10 linhas após remover quantidade 0 e id_cliente null. Fonte: autor (2026).

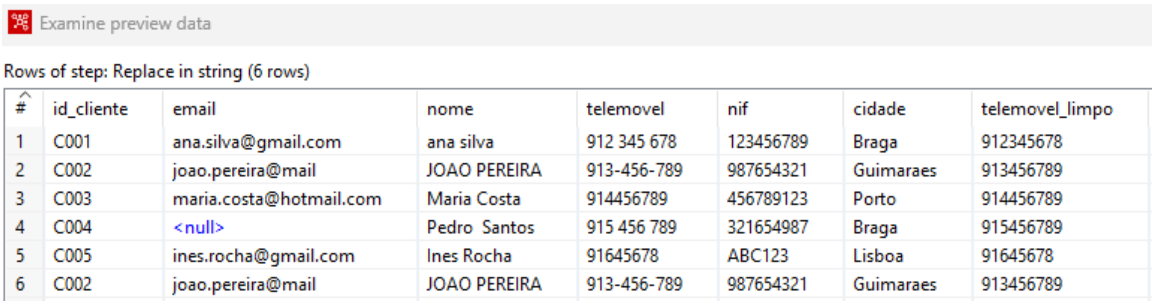


#	id_venda	id_cliente	id_produto	data	quantidade	preco_unit	total_linha
1	V001	C001	P001	2026-10-01	2	899.99	1799.98
2	V002	C002	P002	2026-10-02	1	29.9	29.9
3	V003	C003	P001	2026-10-03	1	899.99	899.99
4	V004	C002	P003	2026-10-04	3	15.5	46.5
5	V006	C001	P003	2026-10-06	5	15.5	77.5
6	V008	C005	P004	2026-10-08	2	49.99	99.98
7	V009	C003	P004	2026-10-09	1	49.99	49.99
8	V010	C001	P005	2026-10-10	1	199.99	199.99
9	V011	C002	P006	2026-10-11	2	79.9	159.8
10	V012	C004	P007	2026-10-12	1	99.99	99.99

Figura 13: Calculator final vendas – 10 linhas com total_linha correto (V001 1799.98). Fonte: autor (2026).

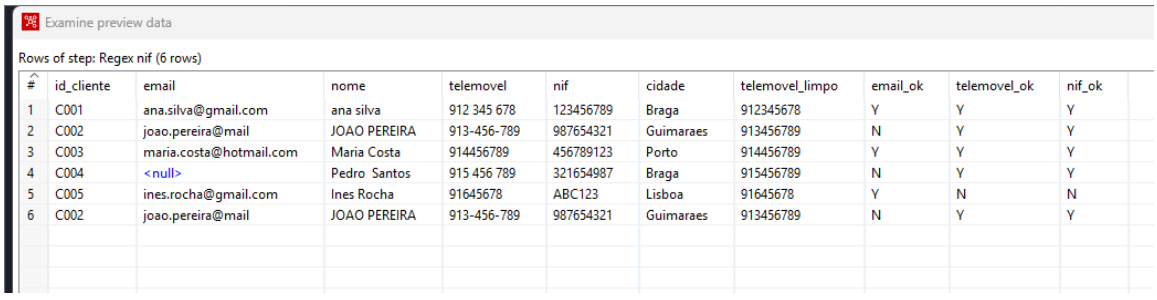
4.3 T_clientes.ktr

Normalizou-se o telemóvel com Replace por expressão regular `[\s-]` (Figura 14), validaram-se correio eletrónico, telemóvel limpo e NIF em três avaliações Regex em série (Figura 15) e filtraram-se os triplos Y/Y/Y, restando 2 limpos (C001, C003), conforme a Figura 16. Aplicou-se anonimização para conformidade com o RGPD (Figura 17).



#	id_cliente	email	nome	telemovel	nif	cidade	telemovel_limpo
1	C001	ana.silva@gmail.com	ana silva	912 345 678	123456789	Braga	912345678
2	C002	joao.pereira@mail	JOAO PEREIRA	913-456-789	987654321	Guimaraes	913456789
3	C003	maria.costa@hotmail.com	Maria Costa	914456789	456789123	Porto	914456789
4	C004	<null>	Pedro Santos	915 456 789	321654987	Braga	915456789
5	C005	ines.rocha@gmail.com	Ines Rocha	91645678	ABC123	Lisboa	91645678
6	C002	joao.pereira@mail	JOAO PEREIRA	913-456-789	987654321	Guimaraes	913456789

Figura 14: Replace telemóvel – normalização com Regex [\s-] (912 345 678 para 912345678). Fonte: autor (2026).



#	id_cliente	email	nome	telemovel	nif	cidade	telemovel_limpo	email_ok	telemovel_ok	nif_ok
1	C001	ana.silva@gmail.com	ana silva	912 345 678	123456789	Braga	912345678	Y	Y	Y
2	C002	joao.pereira@mail	JOAO PEREIRA	913-456-789	987654321	Guimaraes	913456789	N	Y	Y
3	C003	maria.costa@hotmail.com	Maria Costa	914456789	456789123	Porto	914456789	Y	Y	Y
4	C004	<null>	Pedro Santos	915 456 789	321654987	Braga	915456789	N	Y	Y
5	C005	ines.rocha@gmail.com	Ines Rocha	91645678	ABC123	Lisboa	91645678	Y	N	N
6	C002	joao.pereira@mail	JOAO PEREIRA	913-456-789	987654321	Guimaraes	913456789	N	Y	Y

Figura 15: Validação Regex – 6 clientes com email_ok, telemovel_ok e nif_ok (2 limpos C001/C003). Fonte: autor (2026).

ows of step: Filter rows (2 rows)

#	id_cliente	email	nome	telemovel	nif	cidade	telemovel_limpo	email_ok	telemovel_ok	nif_ok
1	C001	ana.silva@gmail.com	ana silva	912 345 678	123456789	Braga	912345678	Y	Y	Y
2	C003	maria.costa@hotmail.com	Maria Costa	914456789	456789123	Porto	914456789	Y	Y	Y

Figura 16: Clientes limpos – 2 válidos após filtro Y/Y/Y com schema remapeado. Fonte: autor (2026).

Examine preview data

Rows of step: Anomaliza (2 rows)

#	id_cliente	nome	email	telemovel	cidade	nif	nif_anon	email_anon
1	C001	ana silva	ana.silva@gmail.com	912345678	Braga	123456789	******789	a***@gmail.com
2	C003	Maria Costa	maria.costa@hotmail.com	914456789	Porto	456789123	******123	m***@hotmail.com

Figura 17: Anonimização RGPD – nif_anon ******789 e email_anon a***@gmail. Fonte: autor (2026).

4.4 T_joins.ktr

Efetuaram-se dois Stream Lookups (sem ordenação prévia, ao contrário do Merge Join): vendas com clientes por `id_cliente` (Figura 18) e resultado com produtos por `id_produto` (Figura 19). Os nulos em C002/C004/C005 refletem integridade referencial, pois esses clientes foram filtrados como sujos. O canvas completo encontra-se na Figura 20 e a agregação por cidade na Figura 21: nulo 429/5, Braga 2072/3 e Porto 948/2 (sem Sort prévio obtinham-se 9 linhas).

Examine preview data

Rows of step: Stream lookup (10 rows)

#	id_venda	id_cliente	id_produto	data	quantidade	preco_unit	total_linha	nome_cliente	cidade_cliente
1	V001	C001	P001	2026-10-01	2	899.99	1798.0	ana silva	Braga
2	V002	C002	P002	2026-10-02	1	29.9	29.0	<null>	<null>
3	V003	C003	P001	2026-10-03	1	899.99	899.0	Maria Costa	Porto
4	V004	C002	P003	2026-10-04	3	15.5	45.0	<null>	<null>
5	V006	C001	P003	2026-10-06	5	15.5	75.0	ana silva	Braga
6	V008	C005	P004	2026-10-08	2	49.99	98.0	<null>	<null>
7	V009	C003	P004	2026-10-09	1	49.99	49.0	Maria Costa	Porto
8	V010	C001	P005	2026-10-10	1	199.99	199.0	ana silva	Braga
9	V011	C002	P006	2026-10-11	2	79.9	158.0	<null>	<null>
10	V012	C004	P007	2026-10-12	1	99.99	99.0	<null>	<null>

Figura 18: Join 1 Stream Lookup vendas+clientes – 10 linhas, nulls por integridade referencial. Fonte: autor (2026).

Examine preview data

Rows of step: Stream lookup 2 (10 rows)

#	id_venda	id_cliente	id_produto	data	quantidade	preco_unit	total_linha	nome_cliente	cidade_cliente	nome_prod	categoria_prod
1	V001	C001	P001	2026-10-01	2	899.99	1798.0	ana silva	Braga	Portatil Lenovo 15	Informatica
2	V002	C002	P002	2026-10-02	1	29.9	29.0	<null>	<null>	Rato Logitech MX	Informatica
3	V003	C003	P001	2026-10-03	1	899.99	899.0	Maria Costa	Porto	Portatil Lenovo 15	Informatica
4	V004	C002	P003	2026-10-04	3	15.5	45.0	<null>	<null>	Cadeira Escritorio	Mobilia
5	V006	C001	P003	2026-10-06	5	15.5	75.0	ana silva	Braga	Cadeira Escritorio	Mobilia
6	V008	C005	P004	2026-10-08	2	49.99	98.0	<null>	<null>	Teclado Mecanico RGB	Informatica
7	V009	C003	P004	2026-10-09	1	49.99	49.0	Maria Costa	Porto	Teclado Mecanico RGB	Informatica
8	V010	C001	P005	2026-10-10	1	199.99	199.0	ana silva	Braga	Monitor LG 27 FullHD	Informatica
9	V011	C002	P006	2026-10-11	2	79.9	158.0	<null>	<null>	Webcam Logitech C920	Informatica
10	V012	C004	P007	2026-10-12	1	99.99	99.0	<null>	<null>	Headset HyperX Cloud	Informatica

Figura 19: Join final Stream Lookup 2 +produtos – 10 linhas com nome_prod e categoria_prod. Fonte: autor (2026).

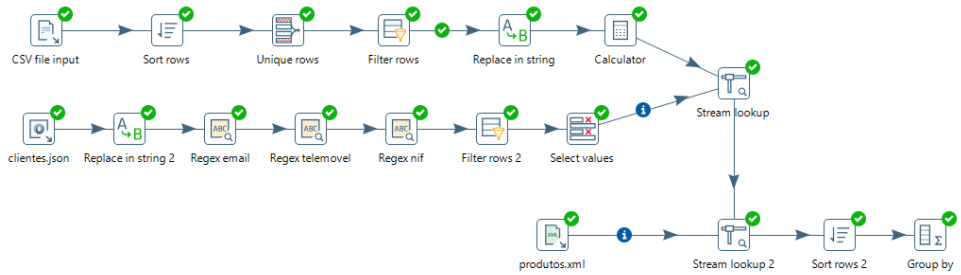


Figura 20: Canvas T_join completo – vendas+clientes+produtos, 2 lookups, Sort+Group. Fonte: autor (2026).

Examine preview data

Rows of step: Group by (3 rows)

#	cidade_cliente	total_cidade	num_vendas
1	<null>	429.0	5
2	Braga	2072.0	3
3	Porto	948.0	2

Figura 21: Group By cidade_cliente – 3 linhas: null 429/5, Braga 2072/3, Porto 948/2. Fonte: autor (2026).

5 Jobs

O presente capítulo descreve a orquestração do processo. A Secção 5.1 apresenta o *Job* principal, a Secção 5.2 a persistência e exportação e a Secção 5.3 a ferramenta complementar.

5.1 J_principal.kjb

Implementou-se o *Job* com a sequência START, E_extracao, T_joins, L_carga, FTP_csv, FTP_json, Mail e Write to log, conforme a Figura 22, com execução integral válida. Utilizaram-se *entries* Transformation para as três transformações, Send Mail para o alerta com os totais 2072/948/429 (Figura 23), FTP Put com um step por ficheiro (limitação da versão 10.2 a um wildcard por step) para servidor local `127.0.0.1:21` e Write to Log para registo de sucesso.



Figura 22: Job principal – E, T, L, FTP csv/json, Mail e Write to Log com sucesso. Fonte: autor (2026).

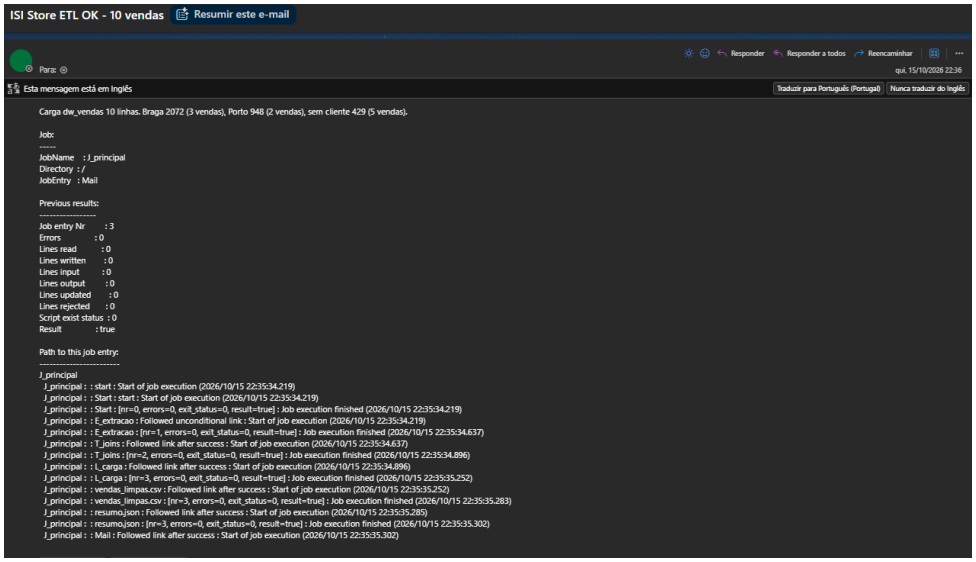


Figura 23: Correio eletrónico de alerta com totais por cidade. Fonte: autor (2026).

5.2 Persistência e exportação

A carga persistiu 10 linhas em `dw_vendas` (Figura 24) e gerou `vendas_limpas.csv` com delimitador `;` e `resumo.json` (Figura 25), ambos transferidos para a pasta `FTP_remoto` que simula o servidor remoto.

Query Query History

```
1  SELECT id_venda, quantidade, preco_unit, total_linha
2  FROM dw_vendas ORDER BY id_venda;
```

Data Output Messages Notifications

	id_venda [PK] character varying (10)	quantidade integer	preco_unit numeric (10,2)	total_linha numeric (10,2)
1	V001	2	899.99	1799.98
2	V002	1	29.90	29.90
3	V003	1	899.99	899.99
4	V004	3	15.50	46.50
5	V006	5	15.50	77.50
6	V008	2	49.99	99.98
7	V009	1	49.99	49.99
8	V010	1	199.99	199.99
9	V011	2	79.90	159.80
10	V012	1	99.99	99.99

Figura 24: Tabela dw_vendas com 10 linhas e totais corretos (V001 1799.98). Fonte: autor (2026).

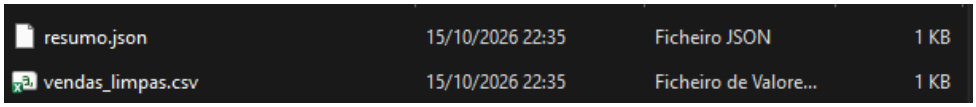


Figura 25: Ficheiros exportados em Dados_Saida. Fonte: autor (2026).

5.3 Node-RED e painel

Como ferramenta complementar configurou-se o *broker* Mosquitto em `localhost:1883` (Eclipse Mosquitto, 2026). O fluxo de *streaming* publica mensagens `STREAM-` com P005 no tópico `isi/vendas` (Figura 26) e o painel apresenta os totais e o gráfico de barras por cidade com os valores do *Group By* (Figura 27), em `.../ui` (Node-RED, 2026).

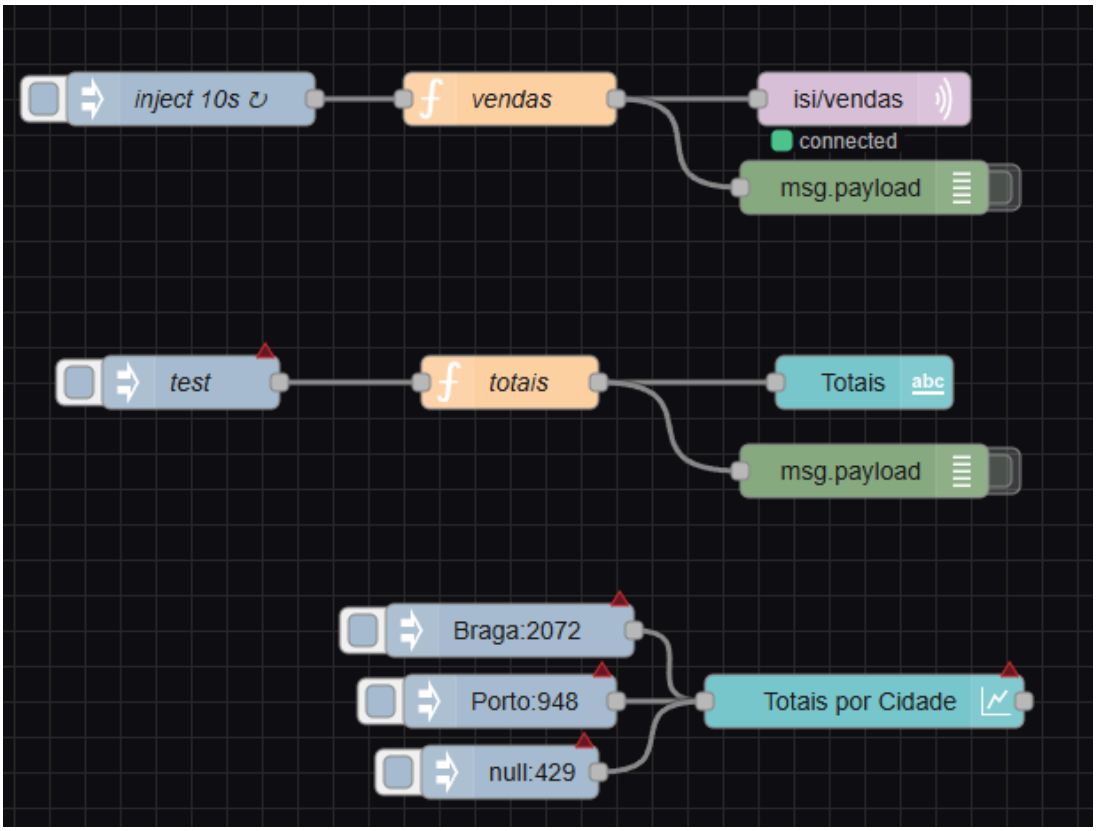


Figura 26: Fluxos Node-RED – streaming isi/vendas e painel. Fonte: autor (2026).



Figura 27: Painel com totais Braga 2072, Porto 948 e nulo 429. Fonte: autor (2026).

6 Vídeo com demonstração (QR Code)

O vídeo com 3 a 5 minutos demonstra a execução do *Job*, os dados de entrada sujos, os resultados limpos, o correio eletrónico e o painel. Disponível em `A-PREENCHER-LINK-YOUTUBE` (não listado).

A-PREENCHER-QR-CODE

7 Conclusão e Trabalhos futuros

Concluiu-se que o processo cumpre as três fases obrigatórias do enunciado (Ferreira e Ribeiro, 2026): extração multiformato validada por contagens (13 vendas, 6 clientes, 158 produtos, 1 REST e 158 stock), transformação com limpeza, expressões regulares, anonimização, junções e agregação, e carga com orquestração completa incluindo correio eletrónico, FTP e registos. Como limitações registam-se o FTP em servidor local por ausência de DNS externo no laboratório e o painel com valores fixos do *Group By*. Como trabalhos futuros propõem-se persistência em MongoDB, painel em Grafana em tempo real e comparação com Apache Airflow.

8 Referências

- Eclipse Mosquitto (2026). *Mosquitto MQTT Broker*. Disponível em `https://mosquitto.org/`.
- FakeStore (2026). *FakeStore API*. Disponível em `https://fakestoreapi.com/`.
- Ferreira, L. e Ribeiro, Ó. (2026). *Enunciado do Trabalho Prático I de ISI*. IPCA.
- Hitachi Vantara (2026). *Pentaho Data Integration Documentation*. Disponível em `https://www.hitachivantara.com/`.
- KNIME (2026). *KNIME Analytics Platform*. Disponível em `https://www.knime.com/`.
- Node-RED (2026). *Node-RED*. Disponível em `https://nodered.org/`.
- PostgreSQL (2026). *PostgreSQL Documentation*. Disponível em `https://www.postgresql.org/`.
- Talend (2026). *Talend Open Studio*. Disponível em `https://www.talend.com/`.

A Contagens de validação

Registaram-se as seguintes contagens ao longo do processo: vendas.csv com 13 linhas, clientes.json com 6 objetos, produtos.xml com 158 produtos, tabela stock com 158 linhas, vendas limpas com 10 linhas, clientes válidos com 2 registos, dw_vendas com 10 linhas e Group By com 3 linhas.